Google Cloud Platform Landing Zone & Organization Foundation (Enterprise)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The problem it solves . . . 4

The approach . . . 4

Why it is safe and governed . . . 4

Solution Overview 5

What this is . . . 5

How it works — the reference design . . . 5

The deployment model — Assess → Design → Configure → Pilot → Validate → Expand . . . 6

Benefits . . . 7

Limitations & boundaries . . . 7

Requirements 8

R1. A bootstrap/seed project and a CI/automation identity . . . 8

R2. Organization ID, billing account ID, and Cloud Identity domain confirmed . . . 8

R3. The agreed resource-hierarchy and naming/labeling design (this solution's design gate) . . . 9

R4. Cloud Identity groups for the IAM baseline . . . 9

R5. Required Google Cloud APIs enabled on the foundation projects . . . 9

R6. A named pilot scope with a defined rollback owner . . . 10

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace . . . 11

2. Organization-level administrative access . . . 11

3. A Cloud Billing account . . . 11

4. The gcloud / IaC toolchain and API enablement . . . 12

5. A resource-hierarchy, naming & environment design . . . 12

6. Security, network & governance stakeholders and policy . . . 12

Implementation 13

Stage 1 — Assess (read-only) . . . 13

Stage 2 — Design . . . 13

Stage 3 — Configure (dry-run first, then apply) . . . 14

Stage 4 — Pilot (already threaded above) → Stage 5 — Validate . . . 16

Stage 6 — Expand . . . 16

Rollback (documented, org-wide changes) . . . 16

Testing & Validation 17

T1 — Hierarchy is present and correctly nested (Gate C1) . . . 17

T2 — Org-policy guardrails are enforced (Gate C2) . . . 17

T3 — IAM baseline is group-based, no user bindings at foundation scope (Gate C3) . . . 18
T4 — Centralized audit logging is flowing (Gate C4) . . . 18
T5 — Shared VPC is attachable and usable (Gate C6) . . . 18
T6 — Billing export is populated (Gate C5) . . . 19
T7 — Naming/labeling conforms (continuous) . . . 19
T8 — Full validation report (Stage 5 gate) . . . 19
Troubleshooting 19
Access & permissions . . . 19
Hierarchy & projects (Gate C1 / Test T1) . . . 20
Org-policy guardrails (Gate C2 / Test T2) . . . 20
IAM baseline (Gate C3 / Test T3) . . . 20
Logging (Gate C4 / Test T4) . . . 21
Billing export (Gate C5 / Test T6) . . . 21
Shared VPC (Gate C6 / Test T5) . . . 21
Tooling . . . 21
Best Practices 22
Hierarchy & naming . . . 22
Guardrails (Organization Policy) . . . 22
IAM & identity . . . 22
Logging, billing & cost . . . 23
Networking . . . 23
Change management & automation . . . 23
Boundaries to respect . . . 24
Deliverables Summary 24
Documentation . . . 24
The 5 Python tools (`scripts/`) . . . 24
Appendix A — PowerShell Scripts 26

Executive Summary

The outcome

The organization gets a **secure, governed Google Cloud foundation** — a landing zone — in place *before* real workloads arrive. Every project that follows lands into a consistent shape: it inherits preventive security guardrails, uses least-privilege access granted through directory groups, feeds a single authoritative audit trail, exports its cost for attribution, and consumes centrally governed networking. The result is a platform where security, cost, and access are correct *by construction*, not reconstructed later under pressure.

The problem it solves

Moving workloads to Google Cloud without a foundation produces the three failures every large migration hits: **sprawl** (uncontrolled projects and networks), **inconsistent security** (each team configures its own, and gaps appear), and **cost you can't attribute** (spend accrues with no owner). Fixing these after workloads land is expensive and disruptive. This solution installs the foundation first, so those failures never accumulate.

The approach

The build follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback for every org-wide change. It stands up the resource hierarchy (Organization → folders → projects), applies a versioned set of Organization Policy guardrails, sets a group-based least-privilege IAM baseline, wires centralized audit logging and BigQuery billing export, and establishes a Shared VPC that workload teams attach to. Five safe-by-default Python tools (read-only or dry-run unless explicitly applied) automate assessment, provisioning, guardrail application, validation, and naming/label conformance.

Why it is safe and governed

- **Preventive, not just detective.** Organization Policy stops misconfiguration (external IPs, service-account keys, out-of-region resources) *before* it happens.
- **Pilot-first with rollback.** Every high-blast-radius change is proven on a named pilot scope and has a documented, owned revert path before it touches the whole Organization.
- **Least privilege at scale.** Access is granted to groups by role and managed in the directory, so access reviews and leaver flows happen in one place.
- **Evidence built in.** One aggregated audit trail and one cost ledger give security and FinOps authoritative, org-wide sources of truth from day one.
- **Verify-first on version-dependent features.** Where a constraint, tier, or preview capability is version-dependent, it is flagged to verify in your organization rather than assumed — and regulatory/legal determinations are deferred to the organization's legal, records, and compliance functions, with this solution supplying the technical method and evidence.

The deliverable is a foundation the enterprise can trust to carry its Google Cloud, Workspace, and Gemini Enterprise ambitions — governed, attributable, and ready to expand.

Solution Overview

What this is

This solution builds the **Google Cloud landing zone**: the secure, governed foundation that every later workload lands on. It is the first thing a large enterprise should stand up when it adopts Google Cloud Platform (GCP) — before any application team creates its first production project.

A landing zone is not a single service you turn on. It is a *coherent set of foundational decisions* made once, applied consistently, and enforced by policy:

• a **resource hierarchy** — the Organization at the root, then **folders** (by environment and business function), then **projects** (the unit of billing, IAM, and API enablement);

• **organization policies** — preventive guardrails that constrain what any project below them can do (allowed regions, no external IPs by default, no service-account key creation, etc.);

• a **least-privilege IAM baseline** delivered through **Cloud Identity / Workspace groups**, so access is granted to *groups by role*, never to individuals ad hoc;

• **centralized audit logging** aggregated into a dedicated logging project, plus **billing export** to BigQuery so cost is attributable from day one;

• a **shared-network landing pattern** (Shared VPC) so workload projects consume vetted, centrally governed networking instead of each team building its own;

• a **naming and labeling convention** so every folder, project, and resource is identifiable, attributable, and queryable.

The enterprise context this is written for: an organization of **80,000–150,000+ users** moving off Microsoft 365 / Entra onto **Google Workspace + Google Cloud Platform**, and adopting Gemini Enterprise. Automation here is **Python** (the `gcloud` CLI and Google Cloud Client Libraries), not PowerShell.

How it works — the reference design

The landing zone is built top-down against the GCP resource hierarchy:

```
Organization (the enterprise domain, e.g. example.com)
■■■ Folder: Common          → shared platform services
■   ■■■ Project: Logging – Common – Prod        (aggregated audit-log sink target)
■   ■■■ Project: Billing-Export – Common – Prod (BigQuery billing export dataset)
■   ■■■ Project: Networking-Host – Shared-VPC – Prod (Shared VPC host project)
■■■ Folder: Production       → live workloads
■   ■■■ (workload service projects attach to the Shared VPC host)
■■■ Folder: Non-Production   → dev / test / staging
■■■ Folder: Bootstrap        → the seed project that provisions the above (IaC state, CI identity)
```

Reference-design decisions this solution makes and names:

1. **Hierarchy.** Folders represent *environment and trust boundary* (Common, Production,

Non-Production, Bootstrap); projects represent *a single deployable/attributable unit*. This keeps org-policy inheritance predictable — a guardrail set on `Folder: Production` applies to every project beneath it.

1. **Guardrails via Organization Policy.** A baseline set of org-policy constraints is applied at

the Organization node (inherited everywhere) and, where a folder needs to relax or tighten, at the folder. The baseline set is named `Org Policy Baseline – Org – v1` and includes commonly recommended constraints such as `iam.disableServiceAccountKeyCreation`, `compute.vmExternalIpAccess`, `compute.skipDefaultNetworkCreation`, `gcp.resourceLocations`, `iam.allowedPolicyMemberDomains` (domain-restricted sharing), and `storage.uniformBucketLevelAccess`. Exact constraint availability and behavior is version-dependent — **verify each constraint in your organization** against the current Organization Policy constraints reference before enforcing.

1. **IAM baseline through groups.** Access is modeled as **roles granted to Google groups**, never

to named users. A small set of foundation groups (e.g. `gcp-organization-admins`, `gcp-billing-admins`, `gcp-network-admins`, `gcp-security-admins`, `gcp-logging-viewers`) is created in the Google Admin console / Cloud Identity, and bound to purpose-built roles at the Organization or folder scope. A **break-glass super-admin** is retained out-of-band, audited, and MFA-protected, and is *not* used for day-to-day work.

1. **Centralized logging.** An **aggregated log sink** at the Organization node routes audit logs

(Admin Activity is always on; Data Access is enabled where policy requires) to a sink target in `Project: Logging – Common – Prod` — typically a log bucket or a BigQuery dataset — so security has one authoritative, tamper-evident audit trail.

1. **Billing export.** The billing account exports **standard usage cost** (and, where enabled,

detailed/pricing exports) to a BigQuery dataset in `Project: Billing-Export – Common – Prod`, so FinOps can attribute spend by folder, project, and label.

1. **Shared-network landing.** A **Shared VPC host project** (`Project: Networking-Host – Shared-VPC

– Prod`) owns the VPC, subnets, routes, and firewall policy; workload (service) projects are *attached* to it and granted use of specific subnets. Teams get networking without owning the network. (VPC Service Controls perimeters, if in scope, are a separate solution that builds on this host.)

1. **Naming & labeling.** Every artifact follows `<Purpose> – <Scope> – <Stage>` for human-facing

names and a normalized `env` / `owner` / `cost-center` / `data-class` **label** set for machine-queryable attribution. A helper tool in this solution generates and validates both.

The deployment model — Assess → Design → Configure → Pilot → Validate → Expand

Every change in this solution follows the house model, pilot-first, with explicit pass/fail gates and a documented rollback for anything org-wide:

• **Assess** — inventory the current Organization, any existing projects, IAM, and billing (the

read-only assessment tool).

• **Design** — agree the hierarchy, naming/labeling convention, guardrail set, group model, and the pilot scope with platform, security, network, and FinOps owners.

• **Configure** — build the hierarchy, apply guardrails, set the IAM baseline, wire logging and billing export, and stand up the Shared VPC — **dry-run first**, then apply.

• **Pilot** — validate on a single named pilot folder/project (`Landing Zone – Pilot`) before touching the whole Organization.

• **Validate** — run the validation report; every gate must pass (hierarchy present, guardrails enforced, logs flowing, billing exporting, Shared VPC attachable).

• **Expand** — onboard remaining folders/projects and workload teams onto the proven pattern.

Benefits

• **Consistency & attributability** — every workload lands into the same governed shape; cost and access are attributable by construction, not reconstructed later.

• **Preventive security** — org policies stop misconfiguration *before* it happens (no external IPs, no service-account keys, region-locked resources), rather than detecting it after.

• **Least-privilege at scale** — group-based IAM means access reviews and joiner/mover/leaver changes happen in one place (the group), not across thousands of bindings.

• **One audit trail, one cost ledger** — centralized logging and billing export give security and FinOps authoritative, org-wide sources of truth.

• **Faster, safer onboarding** — new teams attach to the Shared VPC and inherit guardrails in minutes, with no bespoke networking or policy per team.

Limitations & boundaries

• This is the **foundation**, not the workloads. It does not deploy applications, GKE clusters, data platforms, or Gemini Enterprise itself — it makes the place those land.

• **VPC Service Controls, BeyondCorp Enterprise, and Security Command Center Premium/Enterprise tiers **are referenced as the next layer and are separate solutions; tier-specific features are version-dependent —** verify availability in your organization**.

• **Infrastructure-as-Code** (Terraform / the Cloud Foundation Toolkit blueprints) is presented at the *pattern* level with representative snippets; module internals defer to the tool's own docs. The provided automation is Python over `gcloud` / Cloud Client Libraries.

• **Regulatory and legal determinations** (data-residency law, records-retention duty, legal hold) are deferred to the organization's legal, records, and compliance functions; this solution supplies the technical method (region constraints, log retention, export) and the evidence.

• Exact console navigation and constraint/feature availability change over time; where a step is version- or preview-dependent it is flagged **"verify in your organization"** rather than asserted.

Requirements

The **shared category prerequisites** for every Google Cloud foundation solution are defined in `global-requirements-gcp.md` (in this `source/` folder): (1) a GCP Organization linked to Cloud Identity/Workspace, (2) Organization-level administrative access, (3) a Cloud Billing account, (4) the `gcloud`/IaC toolchain and API enablement, (5) a resource-hierarchy/naming/environment design, and (6) security/network/governance stakeholders and policy. **Read and satisfy those first.** This solution *is* the one that builds much of #5 — so it treats the hierarchy/naming design as an output, not an assumption.

Below are the **solution-specific** prerequisites this landing-zone build adds on top of the shared set. Each lists what it is, why it's required, how to verify it, and how to implement it if missing.

R1. A bootstrap/seed project and a CI/automation identity

• **Description:** A dedicated **seed project** (`Project: Bootstrap – Seed – Prod`) that holds the automation's state (e.g. the Terraform/Cloud Foundation Toolkit state bucket) and a **CI service account** or (preferred) a **Workload Identity Federation** identity that the pipeline uses to build the rest of the hierarchy. Nothing else runs here.

• **Why required:** The landing zone must be built by a controlled, auditable identity — not a human's personal credentials — and its IaC state must live somewhere stable *before* the folders it manages exist. Separating the seed avoids a circular dependency.

• **How to verify:** `gcloud projects describe <seed-project-id>`; confirm a state bucket exists (`gcloud storage buckets list --project <seed-project-id>`) and the CI identity is a member with only the roles it needs (`gcloud projects get-iam-policy <seed-project-id>`).

• **How to implement if missing:** Create the seed project under a bootstrap folder, grant the CI identity org-level roles scoped to what it provisions (e.g. `roles/resourcemanager.folderAdmin`, `roles/resourcemanager.projectCreator`, `roles/orgpolicy.policyAdmin`, `roles/billing.user`), and prefer **Workload Identity Federation** over a downloaded service-account key. **Never** commit or store an exported service-account key.

R2. Organization ID, billing account ID, and Cloud Identity domain confirmed

• **Description:** The exact **Organization ID** (numeric), the **billing account ID** (`XXXXXX-XXXXXX-XXXXXX`), and the **Cloud Identity / Workspace primary domain** the Organization is bound to, captured and agreed with the platform owners.

• **Why required:** Every provisioning step and every tool in this solution keys off these three

identifiers. Domain-restricted sharing (`iam.allowedPolicyMemberDomains`) needs the customer ID of the domain.

• **How to verify:** `gcloud organizations list`, `gcloud billing accounts list`, and (for the domain customer ID) `gcloud organizations describe <ORG_ID> --format="value(owner.directoryCustomerId)"`.

• **How to implement if missing:** Stand up Cloud Identity/Workspace for the domain (per shared requirement #1) so the Organization and its customer ID exist, and create/associate an OPEN billing account.

R3. The agreed resource-hierarchy and naming/labeling design (this solution's design gate)

• **Description:** A written, approved **hierarchy design** (which folders, which environments, which foundation projects) and a **naming + labeling standard** (`<Purpose> – <Scope> – <Stage>` for names; a fixed label key set such as `env`, `owner`, `cost-center`, `data-class`).

• **Why required:** Org policy inheritance, IAM scoping, network attachment, and cost attribution all depend on the shape being decided *before* it is built. Rebuilding a hierarchy after workloads land is expensive and disruptive.

• **How to verify:** The design document is signed off by platform, security, network, and FinOps owners; the naming/labeling helper tool in this solution validates candidate names/labels against it.

• **How to implement if missing:** Run the Design step of this solution — use the naming/label config helper to draft the convention, review with stakeholders, and record it as the approved standard.

R4. Cloud Identity groups for the IAM baseline

• **Description:** A set of **Google groups** in the Google Admin console / Cloud Identity that the IAM baseline binds roles to — e.g. `gcp-organization-admins`, `gcp-billing-admins`, `gcp-network-admins`, `gcp-security-admins`, `gcp-logging-viewers`, plus per-environment `gcp-<env>-developers` groups. Membership is managed in the directory (ideally provisioned from the enterprise HR/IdP source), not in GCP.

• **Why required:** Least-privilege IAM at enterprise scale is only maintainable if access is granted to *groups by role*. Binding roles to individuals produces thousands of unauditable bindings.

• **How to verify:** In the Google Admin console (admin.google.com → Directory → Groups) or via the Cloud Identity / Admin SDK Directory API, confirm the groups exist and are owned by the platform team.

• **How to implement if missing:** Create the groups in the Admin console (or Admin SDK), align them to your role model, and — for an 80k–150k-user enterprise — connect group membership to the authoritative identity source so joiner/mover/leaver flows update access automatically.

R5. Required Google Cloud APIs enabled on the foundation projects

- **Description:** The APIs the landing zone itself depends on: **Resource Manager** (`cloudresourcemanager.googleapis.com`), **Organization Policy** (`orgpolicy.googleapis.com`), **Cloud Billing** (`cloudbilling.googleapis.com`), **Service Usage** (`serviceusage.googleapis.com`), **Compute** (`compute.googleapis.com`, for Shared VPC), **Cloud Logging** (`logging.googleapis.com`), **BigQuery** (`bigquery.googleapis.com`, for billing/log export), and the **Admin SDK / Cloud Identity Groups** APIs where group automation is used.
- **Why required:** Each provisioning action calls the corresponding API; a disabled API produces a `SERVICE_DISABLED` / `PERMISSION_DENIED` failure.
- **How to verify:** `gcloud services list --enabled --project <project-id>` on each foundation project.
- **How to implement if missing:** `gcloud services enable <api> --project <project-id>` for each, ideally as a managed step in the seed/CI pipeline. Enabling some APIs is itself governed by org policy — verify no `serviceuser/gcp.restrictServiceUsage`-style constraint blocks them.

R6. A named pilot scope with a defined rollback owner

- **Description:** A single **pilot folder and/or project** (`Landing Zone – Pilot`) designated as the first place guardrails, IAM, logging, and network attachment are applied — plus a named **rollback owner** authorized to revert org-wide changes.
- **Why required:** The house model is pilot-first. Org-policy and IAM changes at the Organization node are high-blast-radius; proving them on a contained pilot with a ready rollback is mandatory before Expand.
- **How to verify:** The pilot folder/project exists and is agreed; the rollback runbook names an owner and the exact revert commands (see Implementation §Rollback).
- **How to implement if missing:** Create `Landing Zone – Pilot` as a folder with one throwaway project, and document the rollback owner and procedure before applying any org-wide guardrail.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The **Google Cloud Console** (console.cloud.google.com), the `gcloud` **CLI**, and **Cloud Identity / Google Admin** are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability,*

*API, or default is **version- or preview-dependent** (new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is **cited or flagged "verify in your organization"** — never invented. **Infrastructure-as-Code** (Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the **pattern level** — the solutions give the design, the `gcloud`/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as **Python** (Cloud Client Libraries / `gcloud` wrappers), never with hardcoded credentials or keys.*

1. A Google Cloud Organization linked to Cloud Identity / Workspace

• **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

• **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm an Organization resource exists and its ID.

• **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

• **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.

• **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.

• **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the specific roles each solution lists) via `gcloud organizations get-iam-policy`.

• **How to implement if missing:** Have a super-admin grant the roles under change control; keep break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

• **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the solution provisions, with a billing administrator available.

• **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing; FinOps/budget work operates directly on the billing account.

• **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.

• **How to implement if missing:** Create/associate a billing account (Google or reseller) before provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

• **Description:** A workstation or pipeline with the **Google Cloud SDK (**`gcloud`**,** `gsutil`**)**, **Python 3.10+** with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.

• **Why required:** Configuration, automation, and the provided Python tools run through these; most services require their API to be enabled first.

• **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a target project.

• **How to implement if missing:** Install the SDK + libraries; enable the named APIs (`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

• **Description:** An agreed **folder/project structure** (e.g. by department/environment), a **naming/labeling convention**, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

• **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

• **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with the platform owners.

• **How to implement if missing:** Define the hierarchy and naming standard (this is often the first solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

• **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps** stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

• **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to enterprise security and financial policy, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named approvers/owners for security,

networking, and cost.

• **How to implement if missing:** Identify the owners and capture the policy requirements before applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials, exported service-account keys committed to disk, or secrets in code.*

Implementation

Build the landing zone top-down against the resource hierarchy, following the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in later steps, testing, troubleshooting, and the Python tools. **Configure steps are dry-run first, then applied.** Anything that touches the Organization node is proven on `Landing Zone – Pilot` before Expand.

Throughout, set your identifiers once and reuse them:

```
export ORG_ID="123456789012"                # from: gcloud organizations list
export BILLING_ACCOUNT="ABCDEF-123456-GHIJKL" # from: gcloud billing accounts list
export SEED_PROJECT="bootstrap-seed-prod"     # R1
```

> *All `gcloud` shown is current public CLI syntax; where a constraint or feature is version-dependent it is flagged **verify in your organization**. Prefer running provisioning through the CI identity (R1) over a human account.*

Stage 1 — Assess (read-only)

Goal: know exactly what exists before you change anything.

1. **Confirm the Organization and access.**

```
gcloud organizations describe "$ORG_ID"
gcloud organizations get-iam-policy "$ORG_ID" \
  --flatten="bindings[].members" --format="table(bindings.role, bindings.members)"
```

Confirm the account (or CI identity) holds `roles/resourcemanager.organizationAdmin` and the roles R1 lists.

1. **Inventory the current estate.** Run the assessment tool from this solution:

```
python scripts/gcp_org_assessment.py --organization "$ORG_ID" --output org-assessment.json
```

It enumerates folders, projects, existing org policies, billing linkage, and top-level IAM (read-only) and writes a baseline report. **Gate A1:** you can account for every existing project (owner, environment, whether it belongs in the new hierarchy).

1. **Confirm billing.** `gcloud billing accounts list` shows the account OPEN; note the ID.

Stage 2 — Design

Goal: agree the shape once, in writing.

1. **Draft the naming/labeling convention** with the helper:

```
python scripts/gcp_naming_label_helper.py --emit-standard --output naming-standard.json
```

This produces the `<Purpose> – <Scope> – <Stage>` name grammar and the label key set (`env`, `owner`, `cost-center`, `data-class`). Review and adjust with stakeholders.

1. **Agree the hierarchy** (Common / Production / Non-Production / Bootstrap folders and the

foundation projects listed in the Overview reference design) and the **guardrail set** `Org Policy Baseline – Org – v1` with security. **Gate D1:** the design document is signed off by platform, security, network, and FinOps owners, and `Landing Zone – Pilot` is named (R6).

Stage 3 — Configure (dry-run first, then apply)

Build in this order so inheritance and dependencies resolve cleanly. For each provisioning tool, run **without** `--apply` **first** (dry-run prints the planned actions), inspect, then re-run with `--apply`.

3a. Build the folder/project hierarchy

1. **Create the top-level folders.**

```
for F in Common Production Non-Production Bootstrap; do
  gcloud resource-manager folders create --display-name="$F" --organization="$ORG_ID"
done
gcloud resource-manager folders list --organization="$ORG_ID"
```

1. **Provision the foundation projects** with the provisioner (dry-run, then apply). It creates,

under the right folders, the named projects and links billing:

- `Project: Logging – Common – Prod`
- `Project: Billing-Export – Common – Prod`
- `Project: Networking-Host – Shared-VPC – Prod`
- `Project: Bootstrap – Seed – Prod` (if not already created in R1)

```
# dry-run (default: prints plan, changes nothing)
python scripts/gcp_hierarchy_provisioner.py \
  --organization "$ORG_ID" --billing-account "$BILLING_ACCOUNT" --plan hierarchy-plan.json
# apply
python scripts/gcp_hierarchy_provisioner.py \
  --organization "$ORG_ID" --billing-account "$BILLING_ACCOUNT" --plan hierarchy-plan.json --apply
```

Gate C1: every named project exists under the correct folder and is linked to billing (`gcloud billing projects describe <project-id>`).

3b. Apply organization-policy guardrails — pilot first

1. **Apply** `Org Policy Baseline – Org – v1` **to** `Landing Zone – Pilot` **only.** Use the guardrail

applier in dry-run, review, then apply scoped to the pilot folder:

```
python scripts/gcp_org_policy_guardrails.py --scope "folders/<PILOT_FOLDER_ID>" \
  --baseline org-policy-baseline-v1.yaml           # dry-run
python scripts/gcp_org_policy_guardrails.py --scope "folders/<PILOT_FOLDER_ID>" \
  --baseline org-policy-baseline-v1.yaml --apply
```

The baseline includes constraints such as `iam.disableServiceAccountKeyCreation`, `compute.vmExternalIpAccess`, `compute.skipDefaultNetworkCreation`, `gcp.resourceLocations`, `iam.allowedPolicyMemberDomains`, and `storage.uniformBucketLevelAccess`. **Verify each constraint's exact name and behavior in your organization** against the current Organization Policy constraints reference before enforcing — some are boolean, some are list constraints with allow/deny values.

1. **Pilot gate — Gate C2:** on the pilot, confirm the guardrails are *enforced* and that expected

actions are correctly denied (see Testing §T2), e.g. attempting to create a VM with an external IP in the pilot project is denied.

1. **Promote to the Organization node** only after the pilot passes:

```
python scripts/gcp_org_policy_guardrails.py --scope "organizations/$ORG_ID" \
  --baseline org-policy-baseline-v1.yaml            # dry-run
python scripts/gcp_org_policy_guardrails.py --scope "organizations/$ORG_ID" \
  --baseline org-policy-baseline-v1.yaml --apply
```

3c. Set the least-privilege IAM baseline (groups → roles)

1. **Confirm the foundation groups exist** (R4). Bind roles to *groups*, at the narrowest scope that

works:

```
# Example: network admins get network admin only on the Shared VPC host project
gcloud projects add-iam-policy-binding networking-host-shared-vpc-prod \
  --member="group:gcp-network-admins@example.com" --role="roles/compute.networkAdmin"
# Example: security gets org-wide log/SCC viewing at the org node
gcloud organizations add-iam-policy-binding "$ORG_ID" \
  --member="group:gcp-security-admins@example.com" --role="roles/logging.viewer"
```

Grant `roles/resourcemanager.organizationAdmin` only to `gcp-organization-admins`, and keep the **break-glass super-admin** out of daily use (MFA, audited, alarmed on login).

1. **Gate C3:** no role is bound to an individual user in the foundation; the assessment/validation

tool reports zero user-member bindings at the Organization and Common folder scopes.

3d. Wire centralized audit logging

1. **Create the aggregated org-level log sink** into `Project: Logging – Common – Prod`:

```
gcloud logging sinks create org-audit-sink-common-prod \
  "logging.googleapis.com/projects/logging-common-prod/locations/global/buckets/org-audit" \
  --organization="$ORG_ID" --include-children \
  --log-filter='logName:"logs/cloudaudit.googleapis.com"'
```

Grant the sink's writer identity `roles/logging.bucketWriter` on the destination. **Admin Activity** audit logs are always on; enable **Data Access** logs where policy requires (they can be high-volume — scope by service). **Verify Data Access log configuration in your organization.**

1. **Gate C4:** logs appear in the destination (see Testing §T4).

3e. Enable billing export to BigQuery

1. In the Cloud Console, go to **Billing** → **Billing export** → **BigQuery export** and configure

Standard usage cost export into a dataset (e.g. `billing_export`) in `Project: Billing-Export – Common – Prod`. (Billing export to BigQuery is configured in the Console; there is no direct `gcloud` enable command — **verify the current path in your organization**.)

1. **Gate C5:** a `gcp_billing_export_*` table appears in the dataset within ~24h and rows accrue.

3f. Stand up the Shared VPC landing pattern

1. **Enable the host project** and create the VPC/subnets in `Project: Networking-Host – Shared-VPC – Prod`:

```
gcloud compute shared-vpc enable networking-host-shared-vpc-prod
gcloud compute networks create lz-vpc-shared-prod --subnet-mode=custom \
  --project=networking-host-shared-vpc-prod
gcloud compute networks subnets create lz-subnet-prod-use1 \
  --network=lz-vpc-shared-prod --region=us-east1 --range=10.10.0.0/20 \
  --project=networking-host-shared-vpc-prod
```

1. **Attach a service project** (start with the pilot) and grant subnet use to a workload group:

```
gcloud compute shared-vpc associated-projects add <pilot-service-project> \
  --host-project=networking-host-shared-vpc-prod
```

Gate C6: the pilot service project can create a resource on the shared subnet (see Testing §T5).

Stage 4 — Pilot (already threaded above) → Stage 5 — Validate

Run the validation report; **every gate must pass**:

```
python scripts/gcp_landing_zone_validate.py --organization "$ORG_ID" \
  --logging-project logging-common-prod \
  --billing-project billing-export-common-prod \
  --host-project networking-host-shared-vpc-prod \
  --report landing-zone-validation.html
```

The report checks: hierarchy present and correctly nested; `Org Policy Baseline – Org – v1` enforced; IAM baseline has no user-member bindings at foundation scope; the org audit sink is writing; billing export dataset is populated; and the Shared VPC is attachable. **Validation gate:** a PASS on all checks is required before Expand.

Stage 6 — Expand

Onboard the remaining folders/projects and workload teams onto the proven pattern: create service projects under `Folder: Production` / `Folder: Non-Production`, attach them to the Shared VPC, bind the appropriate `gcp-<env>-developers` group, and confirm they inherit `Org Policy Baseline – Org – v1`. Re-run the validation report after each wave.

Rollback (documented, org-wide changes)

The rollback owner (R6) reverts in reverse dependency order. Key reverts:

- **Org policy:** restore prior policy or delete the enforced constraint at the scope it was set:

```
gcloud org-policies delete <constraint> --organization="$ORG_ID"      # or --folder=<PILOT_FOLDER_ID>
```

Because guardrails were **piloted before org-wide apply**, the highest-risk revert is a single folder, not the whole Organization.

- **Log sink:** `gcloud logging sinks delete org-audit-sink-common-prod --organization="$ORG_ID"`.
- **Shared VPC attachment:** `gcloud compute shared-vpc associated-projects remove <project> \ --host-project=networking-host-shared-vpc-prod`.
- **IAM binding:** remove with `remove-iam-policy-binding` mirroring the add.
- **Projects/folders** created in error can be **shut down** (`gcloud projects delete`, recoverable within the 30-day window) / folders deleted once empty.

Each provisioning tool's `--apply` run writes a change record (what it changed, prior value) so the rollback owner has the exact prior state to restore.

Testing & Validation

Each test states its procedure and the **expected pass/fail signal**. Run the pilot tests (§T1–T6) on `Landing Zone – Pilot` before org-wide apply; re-run the full set during Stage 5 — Validate and after each Expand wave. The `gcp_landing_zone_validate.py` tool automates most of these into a single report — the manual procedures below let you confirm any individual gate.

T1 — Hierarchy is present and correctly nested (Gate C1)

- **Procedure:**

```
gcloud resource-manager folders list --organization="$ORG_ID"
gcloud projects list --filter="parent.id=<FOLDER_ID>" --format="table(projectId,parent.id)"
gcloud billing projects describe logging-common-prod
```

- **Pass:** the four folders (Common, Production, Non-Production, Bootstrap) exist; each foundation project sits under its intended folder; each is linked to `$BILLING_ACCOUNT` (billingEnabled = true).
- **Fail:** a folder or project is missing, mis-parented, or not billing-linked → re-run `gcp_hierarchy_provisioner.py` (dry-run to see the diff, then `--apply`).

T2 — Org-policy guardrails are enforced (Gate C2)

- **Procedure (on the pilot):** attempt an action a guardrail should deny.

```
# Should be DENIED if compute.vmExternalIpAccess is enforced
gcloud compute instances create test-ext-ip --zone=us-east1-b \
  --project=<pilot-service-project> --address=  # request an external IP
# Confirm the constraint is set at the scope
gcloud org-policies describe compute.vmExternalIpAccess --folder=<PILOT_FOLDER_ID>
```

- **Pass:** the disallowed action is rejected with a policy-violation error, and `gcloud org-policies describe` shows the constraint enforced at the pilot scope.
- **Fail:** the action succeeds → the constraint is not applied/inherited; re-run `gcp_org_policy_guardrails.py --scope folders/<PILOT_FOLDER_ID> --apply` and re-test before any org-wide promotion. **Verify each constraint name/behavior in your organization** — some

are list constraints needing explicit deny values.

T3 — IAM baseline is group-based, no user bindings at foundation scope (Gate C3)

- **Procedure:**

```
gcloud organizations get-iam-policy "$ORG_ID" \
  --flatten="bindings[].members" \
  --format="value(bindings.members)" | grep -i '^user:' || echo "no user bindings"
```

(Repeat for the Common folder and each foundation project.)

- **Pass:** no `user:` members at the Organization / Common-folder / foundation-project scope — only `group:` (and required `serviceAccount:`) members. `gcp-organization-admins` holds `roles/resourcemanager.organizationAdmin`.
- **Fail:** any `user:` binding appears → replace it with the appropriate group membership. The break-glass super-admin is expected to exist but must be out-of-band, MFA-protected, and alarmed — not used for routine bindings.

T4 — Centralized audit logging is flowing (Gate C4)

- **Procedure:**

```
gcloud logging sinks describe org-audit-sink-common-prod --organization="$ORG_ID"
# generate an auditable event, then read it back from the destination bucket
gcloud logging read 'logName:"cloudaudit.googleapis.com"' \
  --project=logging-common-prod --limit=5 --freshness=1h
```

- **Pass:** the sink exists with `includeChildren=true`, its writer identity has write access to the destination, and recent audit entries are readable in `Project: Logging – Common – Prod`.
- **Fail:** no entries → check the sink writer identity has `roles/logging.bucketWriter`/dataset write on the destination, and that the log filter matches. Data Access logs must be explicitly enabled per service — **verify in your organization**.

T5 — Shared VPC is attachable and usable (Gate C6)

- **Procedure:**

```
gcloud compute shared-vpc get-host-project <pilot-service-project>
gcloud compute networks subnets list --project=networking-host-shared-vpc-prod
# from the service project, create a resource on the shared subnet
gcloud compute instances create lz-smoke-vm --zone=us-east1-b \
  --project=<pilot-service-project> \
  --network-interface=subnet=projects/networking-host-shared-vpc-prod/regions/us-east1/subnetworks/lz-su
```

- **Pass:** the service project reports the correct host; the instance is created on the shared subnet **with no external IP** (consistent with the guardrail).
- **Fail:** attachment or subnet-use fails → confirm the service project is associated and the workload group has `roles/compute.networkUser` on the subnet.

T6 — Billing export is populated (Gate C5)

- **Procedure:** in BigQuery (`Project: Billing-Export – Common – Prod`), query the export dataset:

```
SELECT COUNT(*) AS rows, MAX(usage_end_time) AS latest
FROM `billing-export-common-prod.billing_export.gcp_billing_export_v1_*`;
```

- **Pass:** the `gcp_billing_export_*` table exists and `rows > 0` with a recent `latest` (export begins within ~24h of configuration).
- **Fail:** no table/rows after 24–48h → re-check the Console billing-export configuration points at the correct project/dataset. **Verify the current export path in your organization.**

T7 — Naming/labeling conforms (continuous)

- **Procedure:**

```
python scripts/gcp_naming_label_helper.py --validate \
  --standard naming-standard.json --scan-organization "$ORG_ID"
```

- **Pass:** every folder/project name matches the `<Purpose> – <Scope> – <Stage>` grammar and every project carries the required label keys (`env`, `owner`, `cost-center`, `data-class`).
- **Fail:** the tool lists each non-conforming resource and the expected form → correct names/labels and re-scan.

T8 — Full validation report (Stage 5 gate)

- **Procedure:** run `gcp_landing_zone_validate.py` (see Implementation Stage 5).
- **Pass:** the HTML report shows **PASS** on all checks (hierarchy, guardrails, IAM baseline, log sink, billing export, Shared VPC). This is the gate to proceed to Expand.
- **Fail:** any check FAILs → resolve using the matching test above; do not Expand until the report is all-green.

Troubleshooting

Common failure modes when building the landing zone and how to resolve them. Symptoms are grouped by stage; the named artifacts (`Org Policy Baseline – Org – v1`, `Landing Zone – Pilot`, the foundation projects) match the Implementation and Testing sections.

Access & permissions

`PERMISSION_DENIED` **creating folders/projects or setting org policy.** The identity lacks Organization-level authority. Confirm it holds `roles/resourcemanager.organizationAdmin` (and, for guardrails, `roles/orgpolicy.policyAdmin`; for project creation,

`roles/resourcemanager.projectCreator`) at the Organization node: `gcloud organizations get-iam-policy "$ORG_ID"`. Prefer running through the seed CI identity (R1) with these roles rather than a human account.

`gcloud` **acts on the wrong resource / silently no-ops.** Your active config points elsewhere. Check `gcloud config list` and re-set `gcloud config set project <seed-project-id>`. All examples assume `$ORG_ID`, `$BILLING_ACCOUNT`, and `$SEED_PROJECT` are exported (Implementation top).

Hierarchy & projects (Gate C1 / Test T1)

`gcp_hierarchy_provisioner.py --apply` **fails with** `Project ID already exists`**.** Project IDs are globally unique across all of GCP. Choose a distinct ID (the display *name* can stay, e.g. `Logging – Common – Prod`, but the ID must be unique) and update the plan JSON.

Projects created but `billingEnabled=false`**.** The billing link step failed (often missing `roles/billing.user` on the billing account for the identity). Grant it, then `gcloud billing projects link <project-id> --billing-account="$BILLING_ACCOUNT"`. Re-run Test T1 to confirm.

Folder created twice / duplicated names. Folder display names are not unique. The provisioner checks for an existing folder by display name on `--apply` and skips it; if you created folders manually as well, delete the empty duplicate (`gcloud resource-manager folders delete <folder-id>`) and re-run.

Org-policy guardrails (Gate C2 / Test T2)

A disallowed action still succeeds on the pilot. The constraint isn't enforced at the pilot scope, or its name/semantics differ in your org. Confirm with `gcloud org-policies describe <constraint> --folder=<PILOT_FOLDER_ID>`, then re-run `gcp_org_policy_guardrails.py --scope folders/<PILOT_FOLDER_ID> --apply`. **List constraints** (e.g. `compute.vmExternalIpAccess`, `gcp.resourceLocations`, `iam.allowedPolicyMemberDomains`) need explicit allow/deny values authored in a policy YAML — a boolean-style enforce won't apply them. **Verify the exact constraint name and type in your organization** against the current Organization Policy constraints reference.

`iam.allowedPolicyMemberDomains` **locks out a legitimate identity.** This domain-restricted-sharing constraint denies members outside your customer ID(s). If a needed Google-managed service account or partner domain is blocked, add its customer ID to the allow list (update the value and re-apply), or scope an exception at a lower node. Test on the pilot before org promotion.

Guardrail promotion to the Organization broke an existing workload. This is why the model is pilot-first with a rollback owner (R6). Revert the single constraint at the org scope: `gcloud org-policies delete <constraint> --organization="$ORG_ID"`, restore the prior state from the tool's change log, then re-pilot with the exception encoded before re-promoting.

IAM baseline (Gate C3 / Test T3)

Test T3 reports `user:` **bindings at the foundation scope.** Someone was granted a role directly instead of via a group. Move them into the correct `gcp-*` group and remove the direct binding (`gcloud … remove-iam-policy-binding … --member="user:…"`). The break-glass super-admin is the only

expected individual identity and must stay out-of-band and alarmed — it should not appear in routine role bindings.

Group binding has no effect (members can't do the thing). Confirm the group exists in Cloud Identity (R4) and the user is a member; group membership changes can take a short time to propagate. Also confirm the role was granted at a scope that covers the resource (e.g. network admin on the host project, not just the folder).

Logging (Gate C4 / Test T4)

Sink created but no logs arrive in `Project: Logging – Common – Prod`**.** The sink's **writer service account** lacks write permission on the destination. Get the writer identity from `gcloud logging sinks describe org-audit-sink-common-prod --organization="$ORG_ID"` and grant it `roles/logging.bucketWriter` (log bucket) or dataset write (BigQuery) on the destination. Confirm `includeChildren=true` and that the log filter matches (`logName:"cloudaudit.googleapis.com"`).

Data Access logs missing. Admin Activity logs are always on, but **Data Access** audit logs are off by default and configured per service — enable them where policy requires (they can be high-volume). **Verify the current Data Access log configuration path in your organization.**

Billing export (Gate C5 / Test T6)

No `gcp_billing_export_*` **table after configuring.** Billing export to BigQuery is configured in the **Console** (Billing → Billing export), not via a `gcloud` enable command, and the first rows can take up to ~24h. Confirm the export targets the `billing_export` dataset in `Project: Billing-Export – Common – Prod` and that the BigQuery API is enabled there. **Verify the current export path in your organization.**

Shared VPC (Gate C6 / Test T5)

Service project can't use the shared subnet. Two things must both be true: the service project is **associated** to the host (`gcloud compute shared-vpc associated-projects add <svc> --host-project=networking-host-shared-vpc-prod`) and the workload group/identity has `roles/compute.networkUser` on the specific subnet. Grant subnet use, then re-run Test T5.

Instance creation on the shared subnet is denied for an external IP. That's the guardrail working — `compute.vmExternalIpAccess` denies external IPs. Create the instance with `--no-address` (as Test T5 shows). If a workload genuinely needs egress, use Cloud NAT on the host, not an external IP.

Tooling

A Python tool prints the plan but changes nothing. That's the safe default — the provisioner and guardrail tools are **dry-run unless** `--apply` is passed. Re-run with `--apply` after reviewing the plan (pilot scope first for guardrails).

`gcloud CLI not found on PATH`**.** Install the Google Cloud SDK and ensure `gcloud` is on `PATH`; the tools shell out to it. For `--scan-organization` and any live call, the environment must be authenticated (`gcloud auth login` or ADC). The tools use ADC / the environment — **never** supply a downloaded service-account key.

Validation report FAILs a check you believe is fine. The report checks *presence/enforcement*, not intent. Open the HTML report's Detail column for the exact reason, cross-check with the matching manual test (T1–T7), and remember that some checks (billing export rows, Data Access logs) are flagged verify-in-your-organization because they are Console-configured or version-dependent.

Best Practices

Durable practices for building and extending the Google Cloud landing zone. These outlast the initial build — apply them every time a new folder, project, guardrail, or workload team onboards.

Hierarchy & naming

- **Folders are trust boundaries, projects are attributable units.** Model folders by

environment/trust (Common, Production, Non-Production, Bootstrap), not by team org chart — teams churn, environments don't. Put an org policy where you want it inherited by everything beneath.

- **Decide naming before you build, enforce it forever.** Keep the `<Purpose> – <Scope> – <Stage>`

name grammar and the `env` / `owner` / `cost-center` / `data-class` label set fixed. Run the naming/label helper in CI so a non-conforming project can't land.

- **One purpose per project.** Never co-locate logging, billing export, and networking in a shared

"misc" project — the foundation projects (`Logging – Common – Prod`, `Billing-Export – Common – Prod`, `Networking-Host – Shared-VPC – Prod`) each exist so IAM and blast radius stay separable.

Guardrails (Organization Policy)

- **Pilot every guardrail before org-wide apply.** Apply to `Landing Zone – Pilot`, prove the denial

behavior (Testing §T2), then promote to the Organization node. High-blast-radius changes are never applied blind.

- **Prefer preventive over detective.** An org policy that *prevents* external IPs or service-account

keys is worth more than a finding that reports them after the fact.

- **Verify constraint availability, don't assume it.** The set of Organization Policy constraints and

their allow/deny semantics evolve — check each against the current constraints reference for your organization before enforcing, and record the version you validated against.

- **Keep a documented rollback for every enforced constraint** so the rollback owner can revert a

single scope quickly.

IAM & identity

- **Grant to groups, never to people.** Bind roles to `gcp-*` Google groups and manage membership in

the directory, ideally provisioned from the enterprise IdP so joiner/mover/leaver flows update access automatically.

- **Least privilege at the narrowest scope.** Grant network admin on the host project, not the Organization; grant developer roles on the environment folder, not org-wide.
- **Protect break-glass.** Retain a super-admin out-of-band, MFA-protected, alarmed on sign-in, and excluded from daily operations. Audit its use.
- **No service-account keys.** Enforce `iam.disableServiceAccountKeyCreation` and use Workload Identity Federation / attached service accounts. Never download, commit, or store an exported key.

Logging, billing & cost

- **One authoritative audit trail.** Keep the aggregated org sink (`org-audit-sink-common-prod`) writing to a dedicated logging project with tight IAM and appropriate retention; treat it as security-owned and tamper-evident.
- **Attribute cost by construction.** Because billing exports to BigQuery and every project carries `cost-center`/`owner` labels, FinOps can attribute spend without after-the-fact tagging projects.
- **Set budgets and alerts early.** Add budget alerts per environment folder so runaway spend surfaces before the invoice.

Networking

- **Consume the Shared VPC, don't rebuild it.** Workload teams attach as service projects and get granted specific subnets — no team owns the network. This keeps firewall policy and egress centralized and auditable.
- **Custom-mode VPC, no default network.** Enforce `compute.skipDefaultNetworkCreation` so no project silently gets an unmanaged default network.

Change management & automation

- **Everything through the seed identity, dry-run first.** Provision via the CI/Workload-Identity identity from the seed project; run every tool without `--apply` first and inspect the plan.
- **Keep IaC at the pattern level, source-of-truth in one place.** Terraform / Cloud Foundation Toolkit blueprints express the pattern; keep state in the seed project's state bucket, version the baseline (`Org Policy Baseline – Org – v1`), and bump the version when it changes.
- **Re-validate after every wave.** Run `gcp_landing_zone_validate.py` after each Expand wave — the

landing zone is a living foundation, and drift (a manually created project, a removed constraint) should be caught by the report, not by an incident.

Boundaries to respect

- **Defer legal/regulatory determinations.** Data-residency law, records-retention duty, and legal hold are decided by legal/records/compliance; you provide the technical controls (region constraints, retention, export) and the evidence.
- **Treat tier/preview features as verify-first.** SCC tiers, BeyondCorp Enterprise, VPC Service Controls behaviors, and Gemini Enterprise features are version-dependent — confirm current availability in your organization before you design a dependency on them.

Deliverables Summary

This solution delivers a complete, run-to-implement package for standing up an enterprise Google Cloud landing zone — the governed foundation every later workload lands on.

Documentation

- **Solution Overview** — what the landing zone is, the reference design (resource hierarchy, org policies, IAM baseline, centralized logging, billing export, Shared VPC, naming/labeling), benefits, limitations, and the Assess → Design → Configure → Pilot → Validate → Expand deployment model.
- **Requirements Guide** — the six shared GCP prerequisites plus six solution-specific ones (seed project & CI identity, org/billing/domain IDs, the approved hierarchy/naming design, Cloud Identity groups, required APIs, and a named pilot scope with a rollback owner) — each with how to verify and how to implement.
- **Implementation Guide** — exact ordered steps to build the folder/project hierarchy, apply `Org Policy Baseline – Org – v1` (pilot-first), set the group-based IAM baseline, wire the aggregated audit-log sink and BigQuery billing export, and stand up the Shared VPC — with named artifacts, pass/fail gates, and a documented rollback.
- **Testing & Validation** — eight procedures (hierarchy, guardrails enforced, group-only IAM, logs flowing, Shared VPC usable, billing export populated, naming conformance, full report) each with its expected pass/fail signal.
- **Best Practices** — durable practices for hierarchy, guardrails, IAM, logging/cost, networking, and change management as workloads onboard.
- **Troubleshooting** — common failure modes and resolutions.
- **Executive Summary** — the outcome, approach, and why it is safe and governed.

The 5 Python tools (`scripts/`)

All are **safe-by-default** — read-only, or dry-run unless `--apply` is passed — use Application Default Credentials / the environment (no hardcoded credentials, no exported keys, no secrets in code), and expose `--help`.

1. `gcp_org_assessment.py` — *Assessment / discovery (read-only).* Enumerates the Organization's

folders, projects, existing org policies, billing linkage, and top-level IAM, and writes a baseline inventory report. The starting point for Stage 1 — Assess.

1. `gcp_hierarchy_provisioner.py` — *Hierarchy & project provisioner (dry-run default).* Reads a

hierarchy plan and creates the named folders and foundation projects (`Logging – Common – Prod`, `Billing-Export – Common – Prod`, `Networking-Host – Shared-VPC – Prod`, `Bootstrap – Seed – Prod`), links billing, and records a change log for rollback. Prints the plan and changes nothing unless `--apply` is passed.

1. `gcp_org_policy_guardrails.py` — *Org-policy guardrail applier (dry-run default).* Applies the

`Org Policy Baseline – Org – v1` constraint set to a given scope (pilot folder first, then the Organization), diffing current vs desired and enforcing only on `--apply`. Constraint names are flagged to verify against the current reference for your organization.

1. `gcp_landing_zone_validate.py` — *Validation / report generator (read-only).* Checks all the

gates — hierarchy present, guardrails enforced, IAM baseline group-only, org audit sink writing, billing export populated, Shared VPC attachable — and emits an HTML report with a per-check PASS/FAIL. This is the Stage 5 gate to Expand.

1. `gcp_naming_label_helper.py` — *Naming/label config helper (read-only).* Emits the

`<Purpose> – <Scope> – <Stage>` name grammar and the `env`/`owner`/`cost-center`/`data-class` label standard, and validates candidate names/labels (or scans the Organization) against it — so non-conforming resources are caught in Design and in CI.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

gcp_hierarchy_provisioner.py

gcp_landing_zone_validate.py

gcp_naming_label_helper.py

gcp_org_assessment.py

gcp_org_policy_guardrails.py